```
import { useState, useEffect } from 'react';
import TodoForm from './TodoForm';
import TodoList from './TodoList';
import FilterControls from './FilterControls';
import './App.css';

function App() {
  const [todos, setTodos] = useState(() => {
    const savedTodos = localStorage.getItem('todos');
    return savedTodos ? JSON.parse(savedTodos) : [];
  });
  const [categories, setCategories] = useState(() => {
    const savedCategories = localStorage.getItem('categories');
    return savedCategories ? JSON.parse(savedCategories) : ['Work', 'Personal', 'Shopping'];
  });
  const [filter, setFilter] = useState({
    searchText: '',
    category: 'all',
    status: 'all'
  });

  useEffect(() => {
    localStorage.setItem('todos', JSON.stringify(todos));
    localStorage.setItem('categories', JSON.stringify(categories));
  }, [todos, categories]);

  const addTodo = (text, priority, category, dueDate) => {
    const newTodo = {
      id: Date.now(),
      text,
      priority,
      category,
      dueDate,
      completed: false,
      createdAt: new Date().toISOString()
    };
    setTodos([...todos, newTodo]);
  };
  //Functions
  const toggleTodo = (id) => {
    setTodos(todos.map(todo =>
      todo.id === id ? { ...todo, completed: !todo.completed } : todo
    ));
  };
```

```
  const deleteTodo = (id) => {
    setTodos(todos.filter(todo => todo.id !== id));
  };

  const updateTodo = (id, updatedText) => {
    setTodos(todos.map(todo =>
      todo.id === id ? { ...todo, text: updatedText } : todo
    ));
  };

  const addCategory = (newCategory) => {
    if (!categories.includes(newCategory)) {
      setCategories([...categories, newCategory]);
    }
  };

  const filteredTodos = todos.filter(todo => {
    // Filter by search text
    const matchesSearch = todo.text.toLowerCase().includes(filter.searchText.toLowerCase());

    // Filter by category
    const matchesCategory = filter.category === 'all' || todo.category === filter.category;

    // Filter by status
    let matchesStatus = true;
    if (filter.status === 'active') matchesStatus = !todo.completed;
    if (filter.status === 'completed') matchesStatus = todo.completed;
    if (filter.status === 'overdue') {
      matchesStatus = !todo.completed && todo.dueDate && new Date(todo.dueDate) < new
Date();
    }

    return matchesSearch && matchesCategory && matchesStatus;
  });

  // Sort by priority (High > Medium > Low) then by due date
  const sortedTodos = [...filteredTodos].sort((a, b) => {
    const priorityOrder = { high: 3, medium: 2, low: 1 };
    if (priorityOrder[b.priority] !== priorityOrder[a.priority]) {
      return priorityOrder[b.priority] - priorityOrder[a.priority];
    }

    if (a.dueDate && b.dueDate) {
```

```
      return new Date(a.dueDate) - new Date(b.dueDate);
    }
    return 0;
  });

  return (
    <div className="app">
      <h1>Todo List</h1>
      <TodoForm
        onAdd={addTodo}
        categories={categories}
        onAddCategory={addCategory}
      />
      <FilterControls
        filter={filter}
        setFilter={setFilter}
        categories={categories}
      />
      <TodoList
        todos={sortedTodos}
        onToggle={toggleTodo}
        onDelete={deleteTodo}
        onUpdate={updateTodo}
      />
    </div>
  );
}

export default App;
```

```
import './FilterControls.css';

function FilterControls({ filter, setFilter, categories }) {
  return (
    <div className="filter-controls">
      <input
        type="text"
        value={filter.searchText}
        onChange={(e) => setFilter({...filter, searchText: e.target.value})}
        placeholder="Search todos..."
```

```
        className="search-input"
      />

      <select
        value={filter.category}
        onChange={(e) => setFilter({...filter, category: e.target.value})}
        className="category-filter"
      >
        <option value="all">All Categories</option>
        {categories.map(cat => (
          <option key={cat} value={cat}>{cat}</option>
        ))}
      </select>

      <select
        value={filter.status}
        onChange={(e) => setFilter({...filter, status: e.target.value})}
        className="status-filter"
      >
        <option value="all">All</option>
        <option value="active">Active</option>
        <option value="completed">Completed</option>
        <option value="overdue">Overdue</option>
      </select>
    </div>
  );
}

export default FilterControls;



import { useState } from 'react';
import './TodoForm.css';

function TodoForm({ onAdd, categories, onAddCategory }) {
  const [text, setText] = useState('');
  const [priority, setPriority] = useState('medium');
  const [category, setCategory] = useState('Work');
  const [newCategory, setNewCategory] = useState('');
  const [dueDate, setDueDate] = useState('');

  const handleSubmit = (e) => {
    e.preventDefault();
```

```
  if (!text.trim()) return;
  onAdd(text, priority, category, dueDate);
  setText('');
  setDueDate('');
};

const handleAddCategory = (e) => {
  e.preventDefault();
  if (newCategory.trim() && !categories.includes(newCategory)) {
    onAddCategory(newCategory);
    setCategory(newCategory);
    setNewCategory('');
  }
};

return (
  <form onSubmit={handleSubmit} className="todo-form">
    <input
      type="text"
      value={text}
      onChange={(e) => setText(e.target.value)}
      placeholder="What needs to be done?"
      className="todo-input"
      required
    />

    <select
      value={priority}
      onChange={(e) => setPriority(e.target.value)}
      className="priority-select"
    >
      <option value="high">High</option>
      <option value="medium">Medium</option>
      <option value="low">Low</option>
    </select>

    <select
      value={category}
      onChange={(e) => setCategory(e.target.value)}
      className="category-select"
    >
      {categories.map(cat => (
        <option key={cat} value={cat}>{cat}</option>
      ))}
```

```
      </select>

      <input
        type="date"
        value={dueDate}
        onChange={(e) => setDueDate(e.target.value)}
        className="due-date-input"
      />

      <button type="submit" className="add-button">
        Add
      </button>

      <div className="new-category">
        <input
          type="text"
          value={newCategory}
          onChange={(e) => setNewCategory(e.target.value)}
          placeholder="New category"
          className="new-category-input"
        />
        <button onClick={handleAddCategory} className="add-category-button">
          Add Category
        </button>
      </div>
    </form>
  );
}

export default TodoForm;



import { useState } from 'react';
import './TodoList.css';

function TodoItem({ todo, onToggle, onDelete, onUpdate }) {
  const [isEditing, setIsEditing] = useState(false);
  const [editedText, setEditedText] = useState(todo.text);

  const handleDoubleClick = () => {
    setIsEditing(true);
  };
```

```
const handleEditSubmit = (e) => {
  e.preventDefault();
  if (editedText.trim()) {
    onUpdate(todo.id, editedText);
    setIsEditing(false);
  }
};

const getDueDateStatus = () => {
  if (!todo.dueDate) return '';
  const dueDate = new Date(todo.dueDate);
  const today = new Date();
  today.setHours(0, 0, 0, 0);

  if (dueDate < today && !todo.completed) {
    const diffDays = Math.floor((today - dueDate) / (1000 * 60 * 60 * 24));
    return `Overdue by ${diffDays} day${diffDays !== 1 ? 's' : ''}`;
  } else if (dueDate >= today) {
    const diffDays = Math.floor((dueDate - today) / (1000 * 60 * 60 * 24));
    return `Due in ${diffDays} day${diffDays !== 1 ? 's' : ''}`;
  }
  return '';
};

return (
  <li
    className={`todo-item ${todo.priority} ${todo.completed ? 'completed' : ''} ${
      !todo.completed && todo.dueDate && new Date(todo.dueDate) < new Date() ? 'overdue' : ''
    }`}
  >
    <div className="todo-content">
      <input
        type="checkbox"
        checked={todo.completed}
        onChange={() => onToggle(todo.id)}
        className="todo-checkbox"
      />

      {isEditing ? (
        <form onSubmit={handleEditSubmit} className="edit-form">
          <input
            type="text"
            value={editedText}
            onChange={(e) => setEditedText(e.target.value)}
```

```
            className="edit-input"
            autoFocus
          />
          <button type="submit" className="save-button">
            Save
          </button>
        </form>
      ) : (
        <span
          className="todo-text"
          onDoubleClick={handleDoubleClick}
        >
          {todo.text}
        </span>
      )}
    </div>

    <div className="todo-meta">
      <span className="todo-category">{todo.category}</span>
      {todo.dueDate && (
        <span className="todo-duedate">{getDueDateStatus()}</span>
      )}
    </div>

    <button
      onClick={() => onDelete(todo.id)}
      className="delete-button"
    >
      ×
    </button>
  </li>
 );
}

function TodoList({ todos, onToggle, onDelete, onUpdate }) {
 return (
  <ul className="todo-list">
    {todos.map(todo => (
      <TodoItem
        key={todo.id}
        todo={todo}
        onToggle={onToggle}
        onDelete={onDelete}
        onUpdate={onUpdate}
```

```
        />
      ))}
    </ul>
  );
}

export default TodoList;
```